

Đánh giá MLOps PaaS theo kiến trúc MLOps tham chiếu

Hiện trạng, khoảng trống và lộ trình Degradation-Aware Continuous Training

Báo cáo kỹ thuật nội bộ

August 27, 2026

Abstract

Báo cáo đánh giá repository MLOps PaaS theo kiến trúc tham chiếu của Amou Najafabadi *et al.*, một systematic mapping study tổng hợp 93 nghiên cứu, 39 thành phần kiến trúc, 56 bước quy trình và các vai trò MLOps. Kết quả cho thấy repository đã có nền tảng tốt cho một MLOps PaaS: control plane đa tenant, registry version bất biến, training/execution backend, đóng gói image, serving gateway, thu thập production event và data-drift workflow. Tuy nhiên, hệ thống hiện gần với biến thể inference service có retraining thủ công hơn là continuous training khép kín: thiếu ground-truth feedback, data-quality/performance monitoring, diagnosis đa tín hiệu, policy quyết định action, evaluation gate và traffic splitting có đo lường. Báo cáo đề xuất một lộ trình nghiên cứu và hiện thực hoá “Degradation-Aware Adaptive Model Maintenance”, trong đó detection, diagnosis và action selection là ba lớp tách biệt; retraining chỉ là một action có điều kiện, không phải phản ứng mặc định.

1 Phạm vi và phương pháp đánh giá

1.1 Nguồn tham chiếu

Paper tham chiếu mô tả MLOps bằng hai góc nhìn: *structure* (các thành phần hệ thống) và *process* (các bước tạo, triển khai, vận hành và bảo trì model). Công trình phân loại 39 thành phần trong sáu nhóm: Data Curation; Storage & Versioning; ML Training; CI/CD; Inference; và Infrastructure & Supporting Services. Nó cũng mô tả feedback loops, các biến thể kiến trúc, cùng vai trò của data scientist, data engineer/steward, ML engineer, DevOps/MLOps engineer, domain expert và ML owner [1].

Paper được dùng như *khung tham chiếu*, không phải một checklist bắt buộc hay một chỉ dẫn thực thi. Một thành phần không xuất hiện trong repo không tự động là lỗi; nó là khoảng trống cần được cân nhắc theo mục tiêu sản phẩm và hướng nghiên cứu.

1.2 Bằng chứng repo và giới hạn

Đánh giá ưu tiên source code, database models, manifests và tests hơn README. Các luồng được kiểm tra gồm upload/build, training, inference, registry/deployment và drift. Các kết luận “thiếu” nghĩa là chưa thấy một contract hay implementation nền tảng trong source được kiểm tra, không có nghĩa là người dùng không thể tự đưa logic đó vào code training của mình.

2 Kết luận điều hành

Repository là một **nền tảng MLOps PaaS mạnh ở vận hành model**, nhưng chưa là một hệ thống **continuous training thích ứng đầu-cuối**. Điểm mạnh nằm ở control plane, versioned artifacts, execution backends, đóng gói/deployment và inference event ingestion. Điểm yếu chính nằm ở vòng phản hồi có nhãn, quality/performance monitoring và decision intelligence giữa alert với retraining.

Về biến thể kiến trúc, implementation hiện gần nhất với **inference service + manual retraining**: mỗi deployment resolve một worker/service theo version, còn retraining do người dùng submit training job. Đây chưa phải inference-engine pool với dynamic model loading, và cũng chưa phải online/continuous training variant.

Table 1: Đánh giá định hướng theo khung kiến trúc tham chiếu (heuristic).

Chiều cạnh	Mức	Nhận định
Nền tảng kiến trúc	7/10	Plane separation, state ownership và workflow nền tảng đã rõ.
Storage, registry, lineage model	7/10	Model version, image digest, artifact và training snapshot khá tốt; data/feature lineage còn thiếu.
Training và CI/CD	7/10	Có training runner, Docker/Argo/Kubeflow, packager và registry; chưa có CT policy/evaluation gate.
Inference và observability	6/10	Có gateway, worker version-specific, event stream, Prometheus và data drift; performance feedback chưa hoàn chỉnh.
Closed-loop lifecycle	4/10	Automatic drift tạo drift run, chưa tạo diagnosis hay retraining request.
Stakeholder/governance process	4/10	Có auth/tenant/audit nền tảng; chưa có approval workflow và role mapping cho maintenance.

3 Đối chiếu cấu trúc kiến trúc

Table 2: Ánh xạ nhóm thành phần MLOps sang repository.

Nhóm tham chiếu	Mức phủ	Bảng chứng hiện có	Khoảng trống/hướng tiếp theo
Data Curation	Một phần	Workspace assets cho code/data; reference asset cho drift; training job snapshot input.	Dataset catalog/version bất biến, quality checker, labeling/ground-truth feedback, feature store và schema contract chưa là capability nền tảng.
Storage & Versioning	Một phần	PostgreSQL là lifecycle truth; S3 chứa assets, training outputs và drift reports; registry có immutable model versions, artifacts, metrics, aliases; image digest sau build.	Code/data asset ở project level có thể latest-only; thiếu Dataset entity/version, feature/config/environment lineage đầy đủ và feedback store.
ML Training	Một phần	Training runner chạy entry point upload; MLflow output; Docker local hoặc Argo/Kubeflow production; successful output được build/register.	Preprocessing/feature pipeline, evaluator, continuous training pipeline và HPO policy chưa được platform quản lý.
CI/CD	Có	Model packager build image từ artifact; registry, deployment task, Kustomize/GitOps, CI validation; immutable image/version contract.	Candidate acceptance chưa có quality, business, resource và risk gate thống nhất trước promotion.

Nhóm chiều	tham	Mức phủ	Bằng chứng hiện có	Khoảng tiếp theo	trống/hướng
Inference		Một phần	Authenticated model-server gateway; version-specific serving worker; Redpanda event; consumer persistence; Prometheus counters/histograms; health checks.	Không có engine pool; prediction event chưa mang confidence dù response gateway có confidence; chưa có shadow/canary traffic split, model comparison runner hay XAI module ở platform level.	
Infrastructure & supporting services		Có	Django/Celery, PostgreSQL, S3, Redis, Redpanda, Prometheus/Grafana, Docker, Argo/Kubeflow, Kubernetes/GitOps và Traefik.	Cần resource/cost observability rõ hơn cho CT, GPU/DCGM metrics và UI time series.	

3.1 Những năng lực đã có và có giá trị để kế thừa

- **Plane và state boundary rõ:** control plane điều phối lifecycle; data plane phục vụ inference; execution plane dùng Docker hoặc Argo/Kubeflow; PostgreSQL và S3 giữ trạng thái bền vững.
- **Version immutability:** build thành công tạo model version với image URI/digest và artifacts; alias chuyển version hỗ trợ promotion/rollback mà không sửa version lịch sử.
- **Production event path:** gateway gửi event tenant/project/version-scoped sang Redpanda; consumer ghi vào `paas_production_logs` và outbox tạo automatic-drift signal một cách transaction-safe.
- **Drift workflow:** monitor gắn version với reference asset; Evidently tạo report; callback cập nhật run theo UUID.

3.2 Các khoảng trống quan trọng cần diễn đạt chính xác

- Consumer schema có cột `confidence`, `latency_ms` và `status_code`, nhưng `send_to_redpanda()` hiện chỉ publish features và prediction. Confidence lấy từ worker được trả cho client nhưng chưa đi hết event contract tới production log.
- Evidently hiện chỉ phân tích data drift, chưa đo F1, precision hay recall trên nhãn production.
- Automatic drift outbox và `request_automatic_drift_runs()` chỉ tạo `DriftRun` sau ngưỡng số bản ghi. Không có lời gọi trực tiếp tới training job, HPO hay promotion.
- Registry alias promotion/rollback là manual pointer update. Frontend comparison trả về metrics diff rỗng và recommendation “unknown”, nên chưa phải champion–challenger evaluation.

4 Đối chiếu quy trình lifecycle

Điều này tạo một kết luận kiến trúc quan trọng: repo đã có *plumbing* cho feedback loop, nhưng chưa có *decision loop*. Bổ sung thêm một pipeline retraining đơn giản sẽ không giải quyết phần thiếu chính, vì data drift không đồng nghĩa model degradation và retrain lại data/code hiện hữu có thể lặp lại lỗi.

Table 3: Trạng thái của các bước lifecycle then chốt.

Bước	Mức	Hiện trạng
Onboard, build, register	Có	Upload/training output → build image → immutable version.
Experiment/train	Một phần	Training job có snapshots và outputs; experimentation metadata phụ thuộc code người dùng/MLflow.
Deploy/serve	Có	Ready build được deploy; gateway xác thực và proxy theo tenant/project/version.
Collect production evidence	Một phần	Features/prediction được persist; cần hoàn chỉnh confidence/latency/status và feedback labels.
Detect degradation	Một phần	Data drift và infrastructure metrics có; prediction drift, quality, performance và slice metrics chưa chuẩn hoá.
Diagnose cause	Thiếu	Chưa có cause candidates, evidence graph, confidence hay policy explanation.
Select action and approve	Thiếu	Alert/run chưa thành RetrainingRequest; không có Auto/Human/Engineering routing.
Retrain / evaluate / promote	Một phần	Có train/build/register/deploy thủ công; thiếu candidate gate, shadow/canary, auto rollback và CT cooldown.

5 Hướng phát triển: Degradation-Aware Adaptive Model Maintenance

5.1 Luận đề và phạm vi

Hướng nghiên cứu được chọn là:

Hệ thống nên quyết định cần thay đổi gì trước khi retrain, dựa trên bằng chứng production, lineage và mức rủi ro; không trigger lại pipeline cũ chỉ vì một alert.

Vòng kín mục tiêu là:

Inference + feedback → evidence windows → degradation event → diagnosis → strategy/approval → CT/HPO → evaluation → shadow/canary → promote/rollback

Phiên bản nghiên cứu đầu tiên nên giới hạn vào ba action path có evidence rõ ràng:

1. **Harmful data drift:** distribution shift cùng performance degradation và labels đủ → Auto CT trong policy giới hạn.
2. **Data quality/schema/freshness incident:** → dừng CT, mở engineering incident; không retrain trên dữ liệu lỗi.
3. **Performance degradation chưa chắc nguyên nhân:** → Human-approved CT hoặc investigation, đặc biệt khi labels trễ/mâu thuẫn.

Code/feature semantics, label policy, architecture redesign và runtime outage nên là Engineering recommendation ở MVP. Configuration degradation/HPO là nhánh mở rộng sau khi ba path đầu đã có benchmark.

5.2 Taxonomy evidence-to-action

Confidence không phải thước đo universal của chất lượng model. Nó chỉ có giá trị làm evidence phụ khi được calibration theo model/version/segment; F1 chỉ có sau khi prediction được join với ground truth hợp lệ. Vì vậy prediction ID và delayed-label handling là điều kiện tiên quyết, không phải tiện ích phụ.

5.3 State model tối thiểu

Table 4: Một policy khởi đầu có thể giải thích được.

Evidence pattern	Diagnosis ưu tiên	Action an toàn
Data drift cao; quality ổn; labels đủ; performance giảm	Distribution shift/training data không còn đại diện	Snapshot data window mới, Auto CT nếu policy, budget và gate cho phép.
Quality violation; schema, range hoặc freshness lỗi	Input/data pipeline issue	No CT; incident cho data/engineering owner.
F1/recall giảm; drift thấp hoặc mâu thuẫn	Concept/label/feature/implementation issue chưa phân biệt được	Human review, slice analysis, kiểm tra labels và lineage.
Confidence/entropy xấu; chưa có labels	Early warning hoặc calibration shift	Tăng sampling/monitoring; không Auto CT chỉ từ confidence.
Latency/error rate xấu	Serving/infrastructure issue	Rollback, autoscale hoặc sửa runtime; không CT.
Drift cao nhưng performance/risk ổn	Benign drift	Warning hoặc no action; tránh unnecessary retraining.

Table 5: Các entity cần bổ sung vào control plane.

Entity	Trường cốt lõi	Mục đích
PredictionRecord Feedback	prediction ID, tenant/project/version, timestamp, prediction, confidence, ground-truth, label timestamp/source	Nối prediction với label trễ; kiểm soát tenant và provenance.
EvidenceWindow	model version, window, baseline, metric values, slices, data quality, policy version	Lưu bằng chứng bất biến theo cửa sổ thay vì chỉ report file.
DegradationEvent	evidence IDs, severity, affected component, deduplication key, status	Chuẩn hoá alert có thể idempotent và audit được.
Diagnosis	ranked cause candidates, confidence, explanation, lineage snapshot	Tách suy luận nguyên nhân khỏi detection.
Retraining Request	strategy, data/code/config version, budget, cooldown, approval status	Là đơn vị điều phối thay cho “DriftRun gọi TrainingJob”.
Evaluation Gate	candidate/champion, thresholds, quality, performance, resource, business checks, decision	Chặn candidate không đủ bằng chứng trước production.
Traffic Experiment	shadow/canary cohort, weights, start/end, guardrails, decision	Ghi lại so sánh production và rollback path.

6 Ranh giới tự động hoá và promotion

Table 6: Ranh giới Auto–Human–Engineering cho MVP.

Mức	Hệ thống được phép làm	Không tự làm
Observe / no action	Tạo event, tăng sampling, theo dõi window tiếp theo.	Không tạo training storm chỉ từ warning.
Auto CT	Chọn data window đã phê duyệt, snapshot, train, evaluate, register candidate.	Không đổi label definition, feature semantics, source code hay architecture.
Human-approved CT	Chuẩn bị recommendation, lineage và candidate plan.	Không tự quyết risk/business trade-off hoặc data adequacy mơ hồ.
Engineering intervention	Tạo issue/audit evidence và chờ Git/PR/CI.	Không tự sửa pipeline, dependency, feature code hoặc runtime.

Promotion nên dùng **shadow + canary** làm protocol chính. Shadow nhận cùng traffic thật nhưng không ảnh hưởng user; canary tăng dần 5%, 10%, 25%, 50% khi guardrails đạt. Blue–green là cơ chế chuyển đổi và rollback nhanh, không phải thay thế cho quan sát chất lượng trên traffic thật. Một gate tối thiểu cần xét offline performance, calibration, latency p95, error rate, data-quality guardrails và performance có nhãn khi labels về. Khi labels trễ, chỉ dùng proxy metrics để tạm giữ/rollback; không kết luận model tốt hơn chỉ từ confidence ngắn hạn.

7 Lộ trình hiện thực và nghiên cứu

Table 7: Lộ trình đề xuất khoảng 20 tuần.

Mốc	Mục tiêu	Deliverables	Tiêu chí hoàn thành
0 (1–2)	Freeze protocol nghiên cứu	Dataset/task, scenario injection, ground-truth cause, baselines, budget, metric và seeds.	Benchmark protocol được version-control trước khi có kết quả cuối.
1 (3–6)	Measurement foundation	Prediction ID, confidence/latency/status event contract, feedback API/store, immutable data snapshot, evidence windows.	Join được prediction–label; truy được model → training → data/code/config/environment.
2 (7–9)	Detection	Data drift, quality, prediction distribution, performance với delayed labels; baseline/threshold/slice.	Stable, benign drift, harmful drift và quality scenarios tạo event đúng, idempotent.
3 (10–12)	Diagnosis + HITL	Rule-based cause scoring, explanation, RetrainingRequest, cooldown/budget, approval/audit UI/API.	Unit/integration tests chứng minh warning không gọi train trực tiếp; mỗi decision có evidence IDs.

Mốc	Mục tiêu	Deliverables	Tiêu chí hoàn thành
4 (13–15)	CT candidate	Dataset builder, constrained HPO (nếu có), train, EvaluationGate, registry candidate.	Candidate có reproducible lineage và bị reject/promote theo policy.
5 (16–17)	Production comparison	Shadow/canary routing, guardrails, cohort logs, automatic rollback và alias promotion.	Có champion–challenger experiment có rollback path.
6 (18–20)	Benchmark/paper	Baselines, ablation, sensitivity, cost/human-effort analysis, artifact release.	Kết quả lặp lại được với bảng, plots, limitations và raw meta-data.

7.1 Backlog theo ownership hiện tại

- **model-server + consumer:** mở rộng event contract với prediction ID, confidence, latency và status; bảo đảm durable hand-off để feedback join được event.
- **control plane, drift và observability:** thêm evidence, feedback, degradation, diagnosis, retraining request và evaluation gate; giữ UUID, tenant scope, callback idempotency và PostgreSQL/S3 state ownership.
- **training-runner + execution backends:** nhận immutable snapshot, dataset window/config policy và xuất standardized metrics/artifacts.
- **registry + deployment + GitOps:** candidate/champion metadata, traffic experiment, weighted routing/rollout và rollback guardrails.
- **web:** evidence timeline, approval inbox, candidate comparison thật và audit trail; thay các placeholder metrics/comparison trước khi coi UI là observability source.

8 Thiết kế thực nghiệm

Để claim nghiên cứu có giá trị, benchmark phải biết *ground-truth cause* của từng scenario, không chỉ ground-truth label của bài toán ML. Khởi đầu nên dùng một hoặc hai task tabular classification/regression để kiểm soát chi phí và tái lập:

- S0 stable (no action), S1 benign data drift (warning), S2 harmful data drift (Auto CT), S3 data quality (engineering/no CT), S4 concept/label change (human CT), S5 training-data aging, S6 configuration degradation, S7 code/feature issue, S8 runtime issue và S9 mixed/ambiguous.
- Mỗi scenario có seed, thời điểm inject, cường độ, affected component, expected diagnosis và expected action; tách timeline baseline, injection, detection, intervention và recovery.
- Baselines: manual retraining, periodic CT, alert-driven CT, always retrain và proposed adaptive CT. Các baseline chính nên có budget compute tương đương; “always retrain” là stress/cost baseline, không phải đối thủ thực tế duy nhất.

Metrics cần bao phủ: detection precision/recall/latency; diagnosis top-k accuracy và confidence calibration; strategy accuracy và unsafe automation rate; recovery/time-to-recovery/slice recovery; unnecessary CT; compute/storage/serving cost; approval burden; lineage completeness; reproducibility và rollback success. Không nên chỉ báo cáo F1 cuối cùng: recovery theo thời gian và chi phí của false retraining mới kiểm chứng được policy có ích.

9 Rủi ro và nguyên tắc kiểm soát

- **Delayed labels:** performance metric đến muộn. Thiết kế phải phân tách proxy monitoring với confirmed degradation; lưu timestamp và source của label.
- **Causal ambiguity:** nhiều cause cùng lúc. Diagnosis trả ranked candidates, evidence coverage và no-action/human fallback; không tuyên bố root cause tuyệt đối ngoài controlled scenarios.
- **Unsafe automation:** dùng policy-as-code, budget/concurrency/cooldown, immutable lineage, approval boundary và rollback availability.
- **Multi-tenancy/privacy:** raw feature/prediction/feedback có thể nhạy cảm. Feedback store và evidence query phải tenant-scoped, retention-aware và tối thiểu hoá dữ liệu lưu.
- **External validity:** synthetic drift đơn giản có thể làm kết quả quá lạc quan. Cần varied intensity, missing/delayed labels, mixed causes và temporal replay trước khi đưa kết luận rộng.

10 Kết luận

Repository đã đủ các mảnh ghép để trở thành experimental platform tốt cho nghiên cứu MLOps: immutable model lifecycle, execution infrastructure, serving, production event ingestion và data drift. Giá trị nghiên cứu tiếp theo không nằm ở việc thêm “auto retrain sau drift”, mà ở lớp decision intelligence có thể giải thích được giữa monitoring và execution. Nếu Phase 0–3 được thực hiện trước và benchmark được freeze sớm, repo có thể hỗ trợ một đóng góp rõ ràng về degradation-aware adaptive model maintenance: duy trì recovery tương đương hoặc tốt hơn trong khi giảm unnecessary retraining, chi phí và unsafe automation.

References

- [1] F. Amou Najafabadi, J. Bogner, I. Gerostathopoulos, and P. Lago, “An architectural perspective on MLOps: Structures, processes, tools, and stakeholders,” *Information and Software Technology*, vol. 193, Art. 108029, 2026. doi: [10.1016/j.infsof.2026.108029](https://doi.org/10.1016/j.infsof.2026.108029).
- [2] MLOps PaaS repository, source inspection of control plane, model server, consumer, Evidently, training, registry, deployment, GitOps manifests, and architecture documentation, accessed August 27, 2026.